

再帰下降パーサ③ — 宣言・型・プログラム全体（最終回）

今日のゴール

宣言・型・関数・`typedef` を実装して `parse_program` を完成させる。講義の全テスト入力（約 100 本の `.c`）で AST が scaffold と完全一致したら、Lexer（F1）とあわせて黒箱の完全な置き換えを宣言する。

1 この回の位置づけ

- フロントエンド発展シリーズの最終回（選択制）。前提は F3
- `ProgramParser` は自分の `F3 StmtParser` を継承する。継承の連鎖は F2（式）→ F3（文）→ F4（宣言・型）で完結する
- F2 で先送りした `sizeof` も、この回で最後のピースとして埋める

2 宣言か、文か — C の文法の最難関

F3 のコラムで予告した問題から始める。ブロックの中の次の 2 行を考える。

```
Point p;    /* 宣言 */
x = 1;      /* 式文 */
```

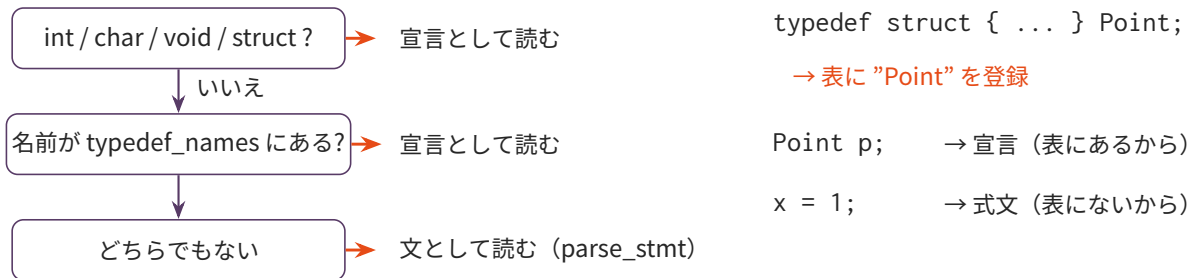
`Point p ;` と `x = 1 ;` は、どちらも「識別子で始まる」。トークンの種類だけを見ても、宣言か式かは決められない。**Point** が型名かどうかを知っていて、はじめて区別できる。

そこでパーサは、`typedef` で定義された型名の集合 `typedef_names` を持ち歩き、「宣言の開始かどうか」を次の順で判定する。

`typedef_names` は解析の途中で育つ。`typedef` を 1 つ読むたびに名前が増え、それ以降の行の解釈が変わる。文法が入力の前の方に依存する — これが C の文法の文脈依存性である。

ブロックの先頭で: これは宣言か?

typedef_names は解析中に育つ



「名前 名前;」という同じ形でも、typedef_names の中身しだい宣言にも式にもなる。
トークンの種類だけでは決められない=Cの文法の文脈依存性

図1 宣言か文か — is_type_start と typedef_names

補足

コラム: lexer hack. language_spec.md のEBNFには「TYPEDEF_NAME は字句解析時にシンボルテーブルを参照してIDENTと区別する」とある。本物のCコンパイラの多くは、実際にパーサから字句解析器へ型名の表をフィードバックする。この折衷は俗に**lexer hack**と呼ばれる。scaffoldは、より単純にパーサ側でTK_IDENTを表と照合する方式を採っている(トークンの種類は変えず、解釈だけ変える)。どちらでも解ける問題だが、「層をまたぐ情報の逆流」が必要になる時点で、Cの文法の設計上の傷と言われる所以である。

3 parse_type — ty_str はここで生まれる

型は `ty_str` 文字列として返す。コマ10からずっと使ってきた `'int'` `'int*'` `'int[5]'` という表記の出所がこの関数である。

```
type ::= base_type { '*' }
```

入力	返す ty_str
int	int
char *	char*
int **	int**

入力	返す ty_str
struct Node *	struct Node*
Point (typedef 済み)	Point

`struct` の後にインライン定義 `{ ... }` が続く場合は、中身を読み飛ばす (`skip_braces`)。フィールドの情報は AST に残さない。

これはコマ 12 の種明かしである。あの回で構造体のフィールドを「ソース文字列を走査して集める」という一見遠回りな方法を使ったのは、Parser が意図的にフィールドを AST に残していないからだ。何を AST に残し、何を残さないかも設計であり、scaffold は「コード生成に必要な最小限」に絞る側に倒している（発展課題で逆の設計も試せる）。

4 宣言 — declarator と配列サフィックス

```
local_decl ::= type declarator [ '=' expr ] ';'
declarator ::= IDENT [ '[' INT ']' ]
```

`int a[5]` は、型 `int` + 宣言子 `a[5]` と読み、`ty_str` を `'int[5]'` に組み立てる。初期化子があれば `parse_expr` で読む（コマ 4 の `int a = 42;` がここに来る）。

ブロックは F3 版を上書きして、C89 スタイル「宣言が先頭」に対応する。

```
def parse_block(self):
    self.expect('{')
    stmts = []
    while self.is_type_start():          # ← この行が F4 の追加分
        stmts.append(self.parse_local_decl())
    while not self.consume_if('}'):
        ...
```

宣言の判定に `is_type_start` を使うので、`typedef` された型のローカル変数（コマ 12 の `Point p;`）もここで正しく宣言になる。

5 sizeof — F2 で先送りした最後のピース

sizeof には 2 つの形がある。

```
'sizeof' '(' type ')'      → ND_SIZEOF_TYPE (ty_str を持つ)
'sizeof' unary_expr       → ND_SIZEOF_EXPR (式を持つ)
```

厄介なのは (を見ただけでは区別できないことである。sizeof(int) は型、sizeof(x) は式 (カッコつきの x)。そこで (の次のトークンを peek(1) で覗き、型の開始なら型版として読む。

これが F2 の LL(1) コラムで「唯一きわどい」と言った箇所である。判定には typedef_names も使うので、sizeof(Node) が型版になるのは typedef struct Node {...} Node; を先に読んでいるからである (コマ 13 の malloc(sizeof(Node)) はこうして解析されていた)。

6 関数 — 宣言か定義かは「読み進めた結果」で分かる

```
func_proto ::= type_declarator '(' [params] ')' ';'
func_def   ::= type_declarator '(' [params] ')' block
```

引数リストを読み終えるまで、宣言 (ND_FUNCPROTO) か定義 (ND_FUNCDEF) かは分からなくてよい。閉じカッコの次が ; なら宣言、{ なら定義と、読み進めた結果で決めればよい (先読み不要)。

引数リストには 3 つの特例がある。

形	扱い
f() / f(void)	引数なし (params は空リスト)
f(char *fmt, ...)	... が来たらそこで打ち切る (可変長宣言。lib.h の printf 用)
引数名の省略 f(int)	名前は空文字列でよい (プロトタイプで使われる)

7 parse_program — 全体を組み立てる

トップレベルは 3 種類だけである。

```
program ::= { typedef | グローバル変数宣言 | 関数宣言/定義 }
```

1. `typedef` なら `parse_typedef()`。**AST ノードは作らず**、型名の登録だけする
2. それ以外は `parse_type` → 名前、と読み進めて、次が (なら関数、そうでなければグローバル変数

グローバル変数（コマ 14 の `int total;` や `int base = 7;`）は、ローカル宣言と同じ `ND_DECL` になる。トップレベルにあるか関数の中にあるかでコード生成側が `.bss/.data` かスタックかを決めていた（コマ 14）。

8 実装

Step	実装対象	内容
1	<code>is_type_start</code> / <code>parse_type</code>	型の判定と <code>ty_str</code> の組み立て
2	<code>parse_array_suffix</code> / <code>parse_local_decl</code> / <code>parse_block</code> （上書き）	宣言。C89 スタイル
3	<code>peek_is_type</code> / <code>parse_sizeof</code>	(の次を覗く
4	<code>parse_func</code>	<code>params · (void) · ... · 宣言/定義</code>
5	<code>parse_typedef</code> / <code>parse_program</code>	型名の登録とトップレベル

`expect_ident` / `skip_braces` / `parse_unary` の上書き（`sizeof` への分岐） / 入口 `parse()` は完成済み。

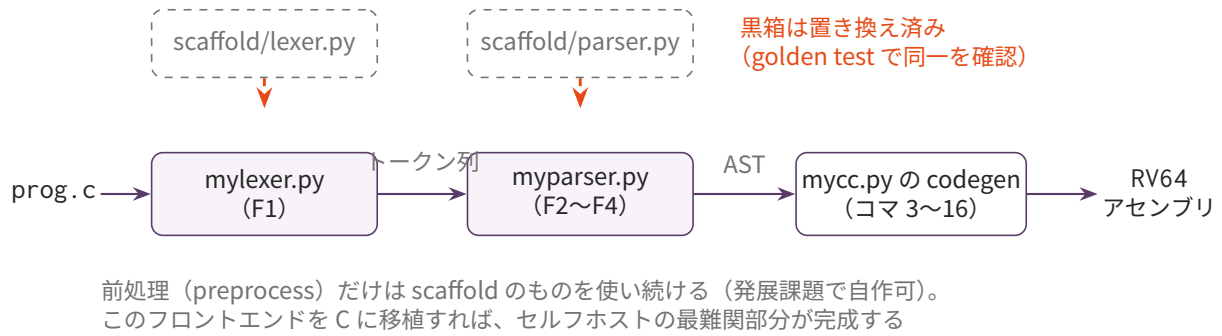


図 2 完成したフロントエンド — 黒箱はもうない

```
python3 myparser.py ../../../../sessions/13_sizeof_malloc_list/tests/list_min.c
```

9 テスト

```
python3 check.py      # Step ごとの単体テスト
python3 golden.py     # 全テスト入力 (約 100 本) で scaffold と突き合わせ
```

check.py の Step 2 以降は、同じ入力を scaffold にも解析させて構造比較する。golden.py が全ファイル一致になったら、このシリーズの完了である。

10 総仕上げ（任意） — mycc.py に差し替えて fixed15 を回す

golden test は「同じ AST が出る」ことの証明なので、理屈の上では自作フロントエンドで fixed15 も通るはずである。実際に確かめたい場合は、final/mycc.py の先頭にある

```
from parser import parse
```

を、自作パーサを読み込む形に差し替える。

```
import importlib.util as _il
_spec = _il.spec_from_file_location(
    "myparser",
    os.path.join(os.path.dirname(__file__), "..", "advanced",
                 "frontend", "F4_parser_decl", "myparser.py"))
```

```
_mod = _il.module_from_spec(_spec)
_spec.loader.exec_module(_mod)
parse = _mod.parse
```

そのまま `python3 scaffold/test_runner.py` を実行して全 PASS すれば、自作の **Lexer/Parser + 自作のコード生成器**でコンパイラ一式が動いたことになる。

11 シリーズのまとめ

回	学んだこと
F0	言語と文法・曖昧性。CYK 法 — 表を埋めれば任意の CFG が解ける (が $O(n^3)$)
F1	字句解析 — 最長一致・分岐の順序・行番号。golden test という検証手法
F2	再帰下降 (式) — EBNF の階層 = 関数の階層。左結合はループ、右結合は再帰
F3	再帰下降 (文) — 先頭トークンでのディスパッチ。dangling else の自然な解決
F4	宣言・型 — 文脈依存性 (typedef_names)。プログラム全体の組み立て

出発点の CYK は汎用だが遅く、到達点の再帰下降は「文法を LL(1) に設計しておけば、先読み 1 トークン・線形時間・手書き可能」だった。この対比が、実用コンパイラのフロントエンドが再帰下降で書かれている理由である。

そして次の一步は Phase 3 の C 移植である。いま Python で書いたこのフロントエンドを C に写せば (dict → 連結リスト、クラス → struct の定石どおり)、セルフホストに必要な部品がすべて自分の手の中に揃う。

12 発展課題

1. 前処理も自作する: `preprocess()` (`#include` / `#define`) を自作して、`scaffold` への依存を完全にゼロにする (上位トラックの `#define` 自前化と同じ課題)
2. **struct** のフィールドを **AST** に残す: `parse_type` の `skip_braces` をやめてフィールドを `ND_DECL` のリストとして持たせ、コマ 12 のソース走査を不要にする改造を設計する (`scaffold` を超える設計変更。 `golden` は通らなくなる)
3. エラー回復: エラーで即座に止めず、`;` か `}` まで読み飛ばして解析を続行し、1 回の実行で複数のエラーを報告する (パニックモード回復)
4. OCaml 参考実装と読み比べる: 配布済みの `workbook/ocaml/support/parser.mly` (Menhir) は、同じ文法を宣言的に書いている。自分の手書き再帰下降と見比べ、生成系が何を自動化しているのかを確認する